Longest Subarray with Equal Number of 0s and 1s

Problem Statement

Given a binary array containing only 0s and 1s, the goal is to find the length of the longest contiguous subarray that contains an equal number of 0s and 1s.

Algorithm Used

The optimized solution uses Prefix Sum and HashMap (Dictionary) to solve the problem efficiently in linear time.

Why Converting 0 to -1 Works

We convert 0 to -1 and keep 1 as +1. If a subarray contains equal numbers of 0s and 1s, then the total sum of that subarray becomes 0. This transforms the problem into finding the longest subarray with sum = 0.

Explanation of the Code

1. A HashMap stores prefix sums and their first occurrence indices.
2. Traverse the array once.
3. Convert 0 to -1 and 1 to +1.
4. Calculate the running prefix sum.
5. If the same prefix sum appears again, then the subarray between those indices has equal 0s and 1s.
6. Update the maximum length accordingly.

Optimized Python Code

```
def findMaxLength(nums):

    prefix_map = {0: -1}

    prefix_sum = 0
    max_len = 0

    for i in range(len(nums)):

        if nums[i] == 0:
            prefix_sum += -1
        else:
            prefix_sum += 1

        if prefix_sum in prefix_map:

            length = i - prefix_map[prefix_sum]
            max_len = max(max_len, length)

        else:
            prefix_map[prefix_sum] = i

    return max_len
```

Dry Run Example

Index	Value	Prefix Sum	Action	Max Length
0	-1	-1	Store index 0	0
1	-1	-2	Store index 1	0
2	+1	-1	Found before at 0	2
3	-1	-2	Found before at 1	2
4	+1	-1	Found before at 0	4
5	+1	0	Found before at -1	6
6	-1	-1	Found before at 0	6

Time Complexity

The algorithm traverses the array once, and each HashMap operation takes O(1) on average. Therefore, the total time complexity is O(n).

Space Complexity

The HashMap may store up to n prefix sums in the worst case. Therefore, the space complexity is O(n).

Why This is the Most Efficient Algorithm

This algorithm is the most efficient because it processes the array in a single traversal without using nested loops. HashMap lookup operations are constant time on average, which allows the algorithm to efficiently detect balanced subarrays. Since every element must be checked at least once, O(n) is the optimal possible time complexity for this problem.